Explorando a intratabilidade e a força bruta no Problema do Caixeiro Viajante

Autor: João Victor Piumbini Cheroto

Disciplina: Projeto e Análise de Algoritmos — Universidade de Vila Velha

Data: 05/11/2026

1. Introdução e objetivo

O Problema do Caixeiro Viajante (TSP) consiste em encontrar um circuito de menor custo que visite cada cidade uma única vez e retorne à origem. Este trabalho examina a solução por força bruta, relacionando a implementação e os resultados experimentais às seções 4.1 e 4.2 da Unidade 04.

O objetivo é verificar o número de rotas examinadas, o custo de avaliação de cada rota e o crescimento do tempo de execução, além de discutir as limitações práticas da busca exaustiva.

2. Implementação e método experimental

O grafo completo é representado por uma matriz simétrica, com diagonal zero e custos inteiros de 1 a 100. A geração utiliza `random.Random(42)`, permitindo reproduzir as instâncias. Os valores representam custos abstratos, sem exigência de desigualdade triangular. Cada tamanho utiliza uma matriz própria, mantida nas repetições; matrizes de tamanhos distintos não precisam representar o mesmo conjunto de cidades.

A cidade de índice 0, correspondente a v_1, permanece fixa. A função `resolver` percorre as permutações dos outros índices, calcula o custo de cada circuito e guarda o menor valor. A função `custo_rota` soma as n arestas, incluindo o retorno à origem. As permutações são geradas sob demanda, sem armazenar todas simultaneamente. Como o enunciado adota $(n-1)!$ rotas, os dois sentidos de um circuito são avaliados mesmo no grafo simétrico.

O experimento solicitou tamanhos de 3 a 13 cidades, com número de repetições igual a 3 por tamanho e limite de 90 segundos por execução. Os tamanhos de 3 a 12 foram concluídos integralmente. A medição usa `time.perf_counter()` dentro do processo de cálculo, incluindo validação da matriz, geração de permutações, avaliação, contagem e escolha do ótimo. A criação da matriz, o início do processo, a comunicação entre processos, a gravação dos arquivos e a geração dos gráficos ficam fora do tempo registrado no CSV.

Os resultados foram obtidos em ambiente em nuvem, com Python 3.12.14 e sistema `Linux-6.18.57-x86_64-with-glibc2.41`. O cálculo é sequencial; a criação de um processo separado serve apenas para permitir o encerramento por tempo. Não foi realizado um teste no computador pessoal do autor. Os tempos dependem das condições desta máquina e da carga do sistema.

Cada repetição concluída tem suas contagens verificadas. O CSV registra os tempos individuais, a mediana, o mínimo e o máximo. Cada tamanho completo é gravado antes do início do seguinte. Uma tentativa interrompida fica registrada em `ambiente.json` e não é apresentada como resultado completo.

3. Resultados experimentais

n	Rotas = (n−1)!	Avaliações de arestas = n!	Mediana (s)	Custo ótimo
3	2	6	0,000019809	101
4	6	24	0,000021172	150
5	24	120	0,000069493	98
6	120	720	0,000094060	157
7	720	5.040	0,000520400	106
8	5.040	40.320	0,003883072	127
9	40.320	362.880	0,037319908	178
10	362.880	3.628.800	0,353710332	144
11	3.628.800	39.916.800	3,686878526	135
12	39.916.800	479.001.600	43,504420408	182

As contagens coincidiram com as expressões teóricas em todas as repetições concluídas. O exemplo de n = 6 corresponde a 120 rotas e 720 arestas, conforme a seção 4.1.7 do material. Na maior instância concluída, n = 12, a rota ótima foi 1-3-6-9-12-4-10-5-2-8-11-7-1, com custo 182. Os custos ótimos entre tamanhos não precisam crescer, pois as matrizes são distintas.

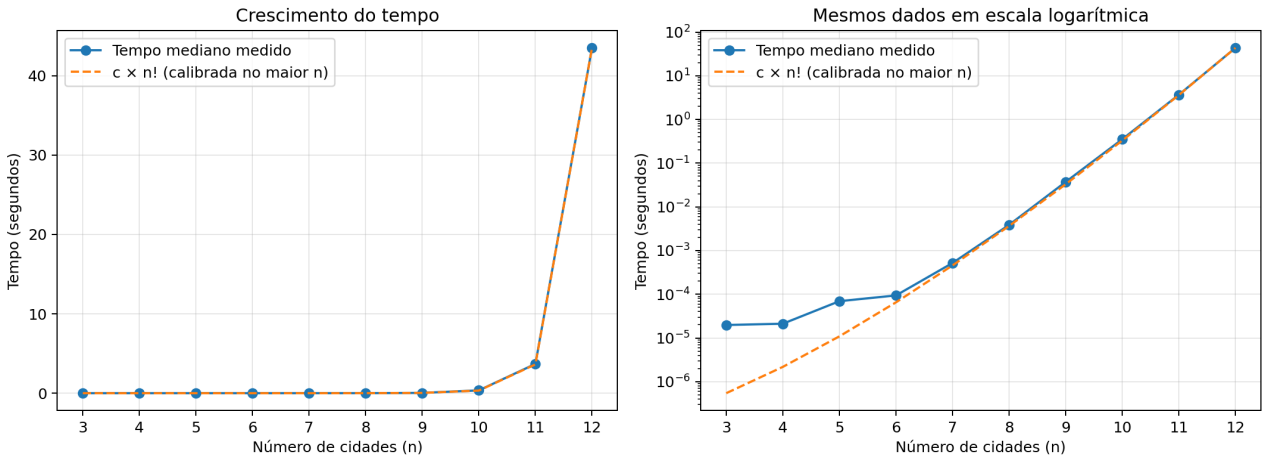


Figura 1: Tempos medidos e curva fatorial, em escalas linear e logarítmica.

Fonte: resultados do experimento registrados em `benchmark.csv`.

A curva de comparação é f(n) = c × n!, com c = 9.08231213e-08 s por avaliação de aresta contada. A calibração utiliza a mediana do maior tamanho concluído, dividida por seu fatorial. A coincidência nesse ponto decorre da calibração; a tendência deve ser examinada nos demais pontos. Para tamanhos pequenos, custos fixos e variações de execução têm maior influência relativa. O experimento é compatível com o crescimento fatorial, enquanto a dedução assintótica decorre da estrutura do algoritmo.

4. Análise e discussão

4.1. Redução do espaço de busca

Sem fixar a origem, seriam geradas n! permutações. Cada circuito orientado apareceria n vezes, uma por possível cidade de partida. Ao fixar v_1, restam (n−1)! rotas. A razão é n!/(n−1)! = n, ou seja, uma redução por um fator n. Para n = 10, a busca passa de 3.628.800 para 362.880 rotas, economia de 90%.

O teste `test_sem_fixacao` compara as duas estratégias em uma instância de cinco cidades: a enumeração sem fixação produz cinco vezes mais rotas e o mesmo custo ótimo. Na implementação principal, apenas os índices de 1 a n−1 são permutados. Para grafos simétricos, eliminar também os circuitos reversos permitiria uma redução adicional por dois; essa otimização não foi aplicada para manter a contagem solicitada.

4.2. Custo de avaliação e complexidade total

O laço de `custo_rota` executa n iterações, consultando e somando uma aresta em cada uma. A ligação da última cidade à primeira também é incluída. Assim, cada avaliação custa $\Theta(n)$, sob o modelo de operações de custo constante para os inteiros utilizados.

Com $R = (n-1)!$ rotas, a quantidade de avaliações de arestas é $A(n) = n \times (n-1)! = n!$. O quociente $A(n)/R$ foi igual a n nos resultados. Portanto, o tempo da implementação é $\Theta(n!)$. A contagem $n!$ não corresponde a todas as instruções da CPU: criação de tuplas, comparações e atualização de contadores também contribuem para o tempo.

A matriz ocupa $\Theta(n^2)$ de memória, enquanto a geração e a avaliação mantêm espaço adicional linear em n. O principal impedimento observado nesta abordagem é o tempo de execução, e não o armazenamento de todas as rotas.

4.3. Limiar prático da intratabilidade

A maior instância concluída foi n = 12, com mediana de 43,504420 segundos. A tentativa com n = 13 foi encerrada pelo limite de 90 segundos na repetição 1. Esse limite considera também o início do processo. A execução foi interrompida sem determinar o ótimo. Não há mediana nem contagem completa para essa tentativa.

O critério de parada estabelece um limite prático para este experimento, não um tamanho máximo absoluto suportado pelo hardware. Uma execução mais longa poderia concluir outra instância.

A tabela seguinte contém somente projeções obtidas com a constante calibrada. Nenhum desses valores deve ser interpretado como um tempo efetivamente medido:

n	Rotas previstas	Avaliações de arestas previstas	Tempo projetado (s)
13	479.001.600	6.227.020.800	565,56
14	6.227.020.800	87.178.291.200	7.917,80
15	87.178.291.200	1.307.674.368.000	118.767,07

Ao acrescentar uma cidade, a contagem de avaliações cresce por um fator n+1. Esse comportamento explica por que poucos acréscimos podem levar a minutos ou horas. O Merge Sort, com $\Theta(n \log n)$, apresenta crescimento muito menor. A recorrência $T(n) = 2T(n-1) + c$, discutida na seção 4.2.2, exemplifica $\Theta(2^n)$; não descreve esta implementação do TSP, cujo crescimento é fatorial.

4.4. P, NP e limites das heurísticas

P e NP são classes de problemas de decisão, com P contida em NP. NP reúne problemas cujas respostas positivas admitem certificados verificáveis em tempo polinomial. Isso não significa que todo problema em NP seja intratável. A versão de decisão do TSP pergunta se existe uma rota de custo no máximo B e é NP-completa. A versão de otimização, abordada neste trabalho, é NP-difícil.

Uma rota candidata permite calcular seu custo e verificar se ele respeita B em tempo polinomial. Esse procedimento não prova que a rota seja ótima. Não se conhece um algoritmo exato polinomial para o TSP geral, e P versus NP permanece em aberto. A análise desta força bruta não demonstra que todo algoritmo exato precise de tempo fatorial. A programação dinâmica, por exemplo, reduz o tempo para $O(n^2 \times 2^n)$, ainda exponencial.

A heurística do vizinho mais próximo pode executar em $O(n^2)$ com uma matriz, mas escolhas locais não garantem o ótimo global. O exemplo abaixo, partindo de A, demonstra essa limitação:

Cidade	A	B	C	D
A	0	1	2	3
B	1	0	4	5
C	2	4	0	100
D	3	5	100	0

O vizinho mais próximo escolhe A→B→C→D→A, com custo 1+4+100+3 = 108. A rota A→C→B→D→A custa 2+4+5+3 = 14. A força bruta confirma que 14 é o ótimo. O exemplo utiliza custos sem a desigualdade triangular, conforme o modelo adotado.

Heurísticas e metaheurísticas podem obter boas soluções em menos tempo, mas geralmente não certificam a otimalidade. Garantias de aproximação dependem do algoritmo e de hipóteses específicas sobre os custos.

5. Validação e reprodução

A suíte contém 24 testes, abrangendo ótimo conhecido, fechamento do circuito, contagem fatorial, economia pela fixação da origem, empates, custos nulos, negativos e fracionários, entradas inválidas e preservação da matriz. Também compara os resultados com uma referência independente por programação dinâmica em 20 matrizes assimétricas, de 3 a 7 cidades.

Os testes do benchmark verificam a gravação dos arquivos, a validação dos argumentos e o encerramento por tempo. Um teste usa uma interrupção simulada para conferir a preservação dos tamanhos anteriores; outro encerra um processo real por limite. Esses testes de infraestrutura são distintos das medições empíricas da seção 3.

Comandos de reprodução, executados na pasta do projeto:

```
python -m pip install -r requirements.txt
python -m unittest -v
python tsp.py --max-n 13 --repeticoes 3 --limite 90
python grafico.py
python relatorio.py
```

No Windows, `python` pode ser substituído por `py`; no Linux, por `python3`. A repetição do experimento altera os tempos. A geração do gráfico e do relatório deve ocorrer após o término do benchmark para manter a correspondência entre os arquivos.

6. Conclusão

A implementação confirmou as contagens de $(n-1)!$ rotas e $n!$ avaliações de arestas nos tamanhos concluídos. A fixação da origem remove redundâncias de rotação, mas

mantém a complexidade $\Theta(n!)$. Os resultados evidenciam a diferença entre a viabilidade em instâncias pequenas e o crescimento do tempo ao aumentar o número de cidades.

O limite adotado e as projeções permitem discutir a intratabilidade prática sem confundir uma execução interrompida com um resultado completo. A busca exaustiva garante o ótimo quando termina; estratégias mais rápidas, como a escolha gananciosa, não oferecem essa garantia em geral.

Referência

UNIVERSIDADE VILA VELHA. **Unidade 04: Projeto e Análise de Algoritmos**. Material disponibilizado na disciplina, arquivo `Unidade_04 (6).pdf`, seções 4.1 e 4.2. Acesso em: 5 nov. 2026.